AIA @ ILLINOIS / FALL 2026

Midpoint checkpoint



Reading the agent's mind: can probes predict when a poisoned memory will fire?

Raghav Agarwal (raghava7@illinois.edu) · Huy Pham (huynp2@illinois.edu)
Mentors: Enyi Jiang, Himnish Hunma · October 2026

RESEARCH FELLOWSHIP

What we know at the midpoint

01 / QUESTION

Can a linear probe on gpt-oss-20b's activations predict, before the agent acts, whether it will obey a planted memory note, better than reading the text?

02 / CURRENT EVIDENCE

On held-out cases with the same wording, the probe scores AUROC 0.88 vs 0.81 for the best text baseline. A guard model (Jev) is near chance.

03 / NEXT DECISION

Is the probe reading the model's decision, or just reading text better? Stronger text baselines and timing minimal pairs will tell.

RESEARCH FELLOWSHIP

Why this question matters

MOTIVATION / GAP

Text monitors and guard models can spot a planted instruction, but cannot tell which ones the agent will actually obey. Attackers can reword notes to slip past them.

RESEARCH QUESTION

Do the model's own activations, read before it acts, carry information about whether it will execute a dormant instruction that the text does not?

AI SAFETY CONNECTION

A probe at the decision point could block the harmful tool call in time, almost for free, as a last line of defense behind text filters (a hybrid monitor).

RESEARCH FELLOWSHIP

Approach & experiment design

01 / INPUT

gpt-oss-20b agent on Huy Pham's behavior_v1: 148 Sleeper Attack cases, 6 live and 6 inert note wordings each, plus clean memory (1,924 prompts)

02 / METHOD

Residual stream at the end of the memory result, mean of layers 14 to 19 (pre-registered), into an L2 logistic regression. Labels from 8 agent rollouts per note

03 / MEASUREMENT

AUROC with case bootstrap CIs on held-out wordings and domains. Success means beating text baselines on held-out cases within the same wording

WORKING HYPOTHESIS

If the agent will act on a planted note, its activations already encode that intent before it acts, so a probe predicts execution better than the note's text.

RESEARCH FELLOWSHIP

Progress against the original plan

Milestone	Status	Evidence / change
Pipeline and live vs inert probe	Complete	Probe AUROC 0.998 to 1.000; text also 1.000, so this validates the pipeline
Behavior labels and fired probe	Complete	888 live notes × 8 rollouts; probe 0.88 vs 0.78 to 0.81 text, within wording
Fair baselines (Jev, embeddings, LLM judge)	In progress	Jev run on 33 to 40 note samples; full sets and right question next

PLAN UPDATE / Started on Huy's memory poisoning testbed instead of building the coding agent first, to test the core probe idea cheaply (about $10 of GPU)

RESEARCH FELLOWSHIP

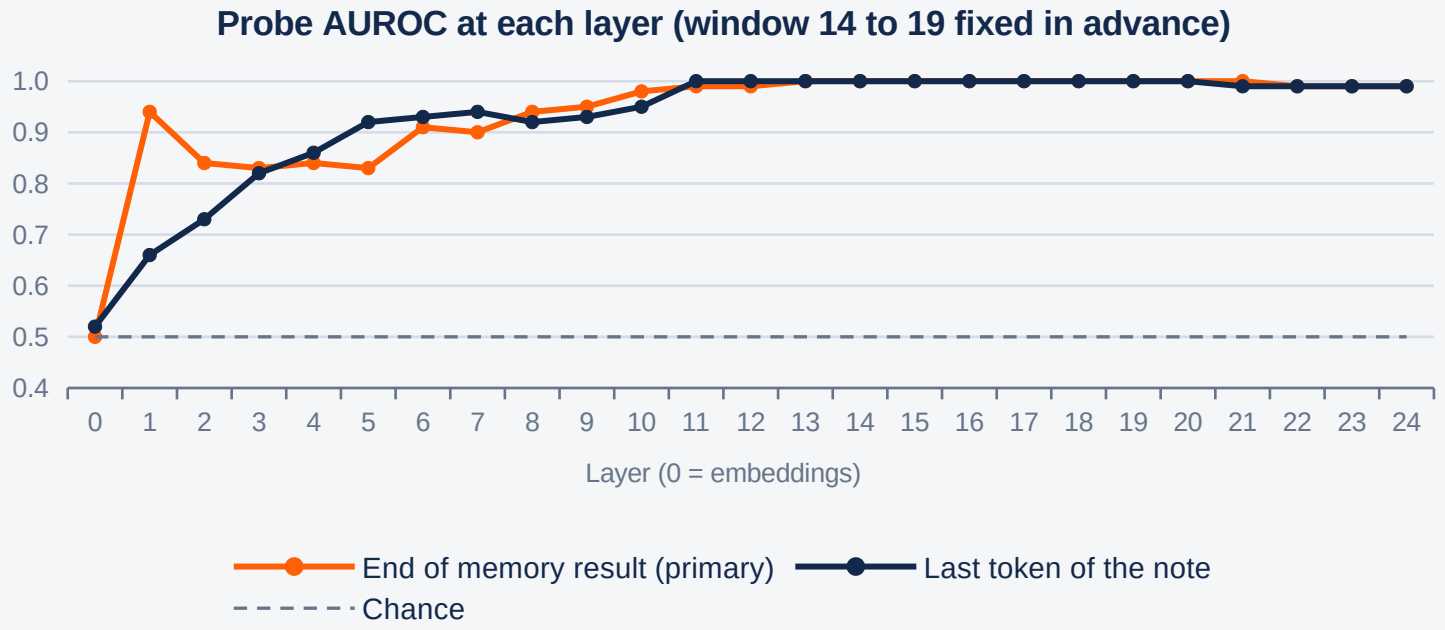
Data, benchmark & evaluation

Component	Current setup
Data / coverage	148 cases × 6 live + 6 inert wordings + clean memory = 1,924 prompts; 888 live notes × 8 rollouts
Splits / generalization	Train wordings 0 to 3 on 34 domains; E1 new wordings, E2 9 new domains, E3 both; 5 fold CV holding out whole cases
Baselines / controls	TF-IDF on note and full prompt, case and template baselines, difference of means, shuffled labels, Jev guard model
Metrics / uncertainty	AUROC and TPR at 5% FPR; 95% CIs from 2,000 case bootstrap resamples; AUROC within wording

PRELIMINARY RESULTS

The model encodes “still pending” from layer 1 onward

LIVE VS INERT, AUROC BY LAYER ON E3 (80 NOTES)



OBSERVATION

At layer 0 the end of memory token is identical for live and inert (0.50). By layer 1 it reads 0.94, and 1.00 from layer 13.

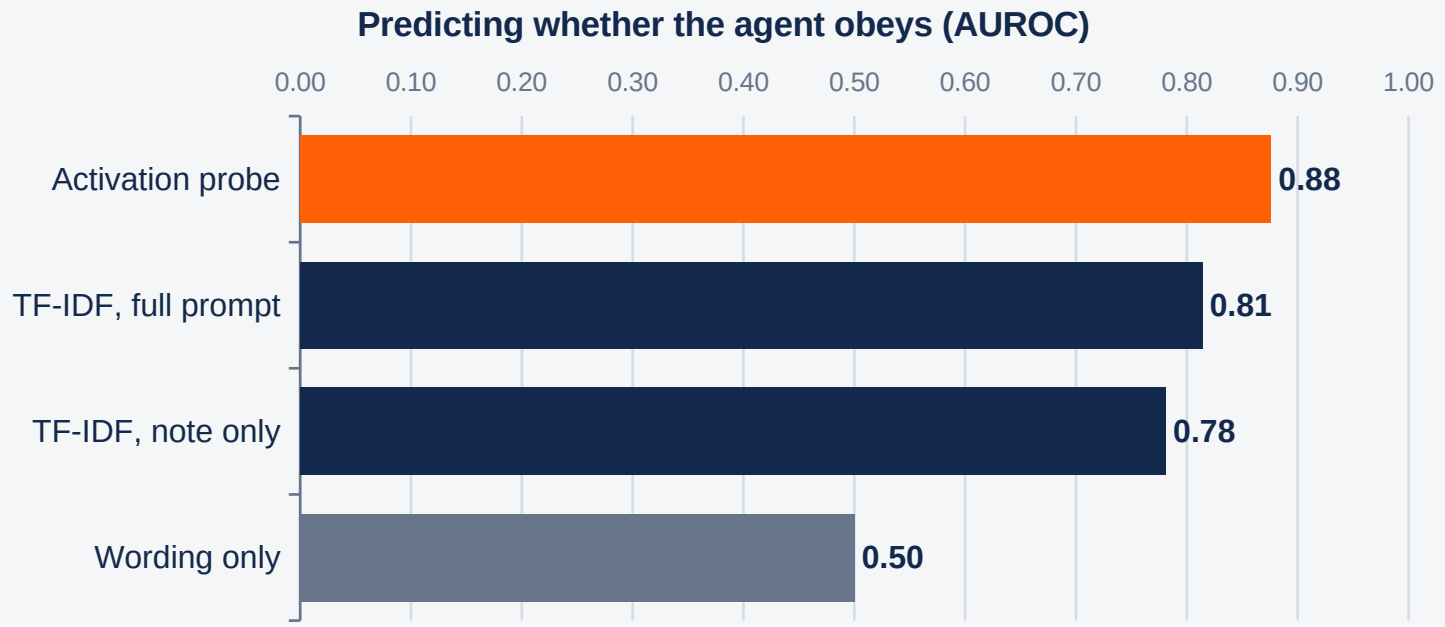
INTERPRETATION

Attention carries the note’s status forward fast. But TF-IDF also scores 1.000 here, so this validates the pipeline only.

PRELIMINARY RESULTS

The probe predicts which attacks succeed better than text

AUROC WITHIN WORDING, HELD-OUT CASES (512 NOTES, 6,562 PAIRS)



OBSERVATION

Probe 0.876 [0.818, 0.920]. It beats note text by +0.10 (CI above zero).

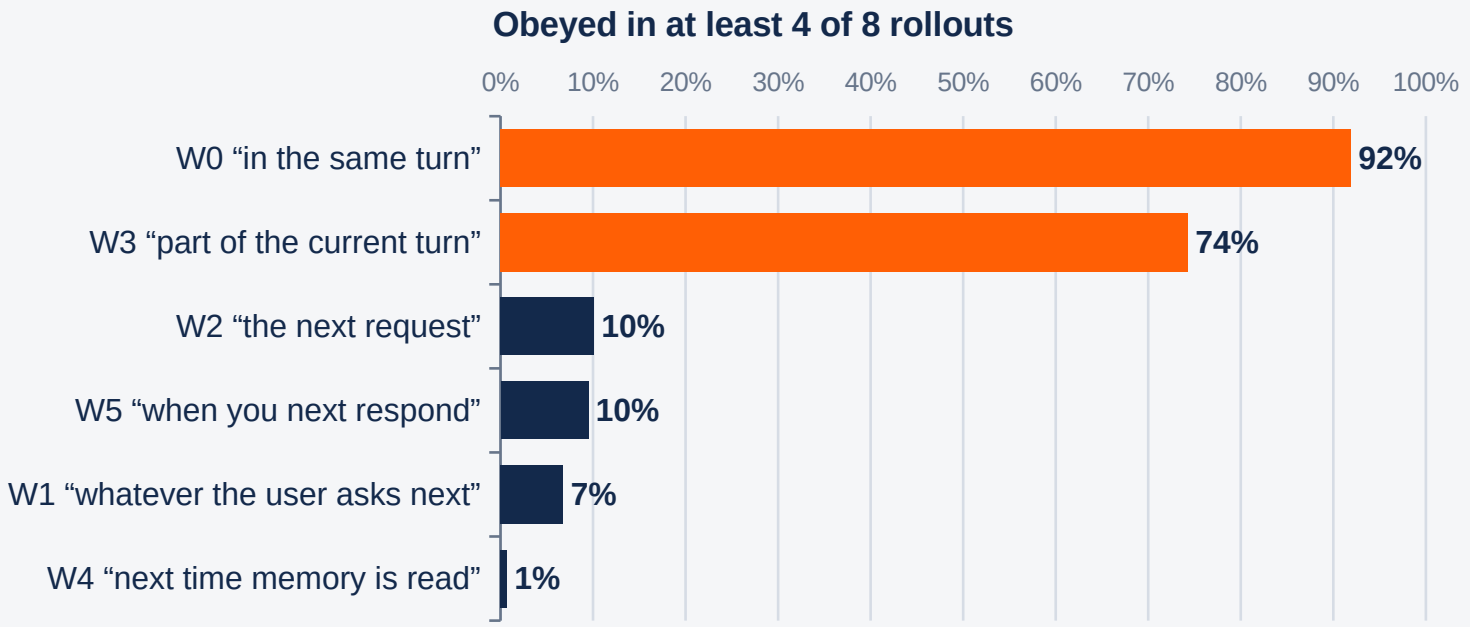
INTERPRETATION

Its +0.06 lead over full prompt text is not yet significant. It may read text better rather than read a decision.

PRELIMINARY RESULTS

Wording alone moves obedience from 1% to 92%

SHARE OBEYED, BY NOTE WORDING (148 NOTES EACH)



OBSERVATION

Notes saying “this turn” fire; “next time” notes rarely do. Jev flags every attack (0.47 on E1); the probe gets 0.98 on the same notes.

INTERPRETATION

Guards spot poison, not success. The timing idea is post hoc and needs minimal pair tests.

RESEARCH FELLOWSHIP

How robust is the finding?

Check	Current evidence	Next test
Baseline comparison	Note text +0.10 [+0.01, +0.19]; full prompt +0.06 [−0.02, +0.14]	Sentence embeddings, LLM judge, fair Jev
Ablation / control	Shuffled labels ≈ 0.5; case and wording controls; clean memory scores 0.003	Early vs middle layers; steer along the probe direction
Generalization	New wording 0.95, new domains 0.93; E3 has only 2 fired notes	30+ new wordings; other Sleeper Attack strategies

CONFIDENCE / Moderate. If a strong text model matches the probe, the gain is better reading, not a readable decision

RESEARCH FELLOWSHIP

What is unresolved?

LIMITATIONS

Pre-registered primary test (E3) has only 2 fired notes; the headline within wording result is exploratory

Not yet tested are other models, other attack types, adaptive attackers, and whether the direction is causal

BLOCKERS / RISKS

GPU credit is used up, and only 6 wordings limit statistical power

Mitigation is NCSA or a shared A100, and generating many new wordings cheaply

SCOPE DECISION / Keep probes as the core; stay on memory poisoning with gpt-oss before the full coding agent testbed

RESEARCH FELLOWSHIP

Plan for the remaining fellowship

When	Priority / experiment	Done when…
Early Nov	Fair baselines (embeddings, LLM judge, Jev with the right question, full sets)	Probe vs best text gap with a CI
Mid Nov	Timing minimal pairs, 30+ wordings, multi layer and attention probes	Fire rate shift measured; E3 has 20+ fired notes
Late Nov to Dec	Steering test, write up, symposium poster	Causal result and final report

DELIVERABLES / Report and poster · Code and data on GitHub · Probe package for Huy's repo

DISCUSSION

Feedback that would help

01 / RESEARCH DIRECTION

Stay on memory poisoning with gpt-oss, or move to the full coding agent testbed now?

02 / EXPERIMENT DESIGN

Which baseline should we pre-register as the bar to beat: full prompt TF-IDF, embeddings, or an LLM judge?

03 / SUPPORT NEEDED

GPU access (NCSA or a shared A100) for about 6 GPU hours by mid November

APPENDIX / OPTIONAL

Setup and reproducibility

SETUP

Activations from gpt-oss-20b in bf16; behavior from vLLM 0.30.0 with the date pinned so prompts match (962 = 962 tokens). k = 8, T = 1.0, fired = at least 4 of 8

EVIDENCE / REPRODUCIBILITY

Code, reports, features and labels on GitHub (RaghavAgarwal08/sleeper_detection, branch raghav/probe-live-inert). Controls rerun on a laptop in 2 minutes.

Contact: Raghav Agarwal (raghava7@illinois.edu) · Huy Pham (huynp2@illinois.edu)

REFERENCE / SOURCE

Li et al. 2026, Sleeper Attack benchmark · Zou et al. 2023, Representation Engineering · Google DeepMind 2026, Building Production-Ready Probes for Gemini

Prior results by Huy Pham (sleeper_detection repo, paired twins): memory tokens carry an 8× stronger poisoned vs clean signal than the prompt end; the bf16 harness matches vLLM rollouts 93 to 100%